HW04

Sam Ly

March 9, 2026

Exercise 1 [2]

Tools and resources used: Mainly Google, and Gemini when I got really stuck.

Exercise 1 Points attempted: 2. Time spent: 10 minutes. Difficulty: 1/10. Self-assessed score: 2/2. Questions/Feedback: None.

Exercise 2 Points attempted: 7. Time spent: 120 minutes. Difficulty: 7/10. Self-assessed score: 7/7. Questions/Feedback: Correctness

Exercise 3 Points attempted: 1. Time spent: 5 minutes. Difficulty: 1/10. Self-assessed score: 1/1. Questions/Feedback: Correctness

Choice Exercise 6 Points attempted: 10. Time spent: 5 minutes. Difficulty: 8/10. Self-assessed score: 10/10. Questions/Feedback: Correctness

Exercise 2 [7]

1. We defined the dihedral group in terms of its group presentation:

$$D_n = \langle r, f \mid r^n = f^2 = (rf)^2 = e \rangle.$$

(a) Prove that $r^k f = f r^{n-k}$ for all $0 < k < n$.

Proof. First, we notice that $rfrf = e$. By right muliplying by f we get $rfr = f$. By left muliplying by r^{-1}, we get $fr = r^{-1}f$. Then, we left and right muliply by f on both sides to get $rf = fr^{-1}f$.
In other words, $r^1 f = f r^{n-1}$. Now, to prove that $r^k f = f r^{n-k}$ for all $0 < k < n$, we must use induction. We have already proved the base case $k = 1$. As our inductive hypothesis, we assume that the proposition is true for some value k. Thus, $r^k f = f r^{n-k}$. Now, for by left multiplying by r, we get

$$r^{k+1} f = r f r^{n-k}.$$

Now, we apply $rf = fr^{n-1}$ to the RHS, and get

$$r^{k+1} f = f r^{-1} r^{n-k} = f r^{n-(k+1)}.$$

Therefore, $r^k f = f r^{n-k}$ for all $0 < k < n$. □

(b) Prove that every element can be written in the form $r^k f^\ell$ for $0 \leq k < n$ and $\ell \in \{0, 1\}$

Proof. We approach this problem from a formal languages angle. We begin by constructing all possible 'words' of the language formed by the alphabet $\{r, s\}$. One example of such a word is $rrrfrfffrffrrrrrrrf$. By definition, this language must encompass all of D_n for any n.
Then, we can form some basic substitution rules, in other words, a 'grammar' that allows us to rewrite the word into a nicer form. By definition of the dihedral group, we have the rules:

$$\underbrace{rr\ldots r}_{m \text{ terms}} = r^m = r^{m-n} \tag{1}$$

$$ff = e. \tag{2}$$

Thus, we can rewrite our word into the form $r^{k_0} f^{l_0} r^{k_1} f^{l_1} \ldots$, where $0 \leq k_i < n$ and $l_i \in \{0, 1\}$.
Now, from here, we apply the proposition proved above to combine and move all l_i to the right of the expression.
For example, if $l_0 = 0$, then the expression becomes $r^{k_0} r^{k_1} f^{l_1} \cdots = r^{k_0 + k_1} f^{l_1} \ldots$. If $l_0 = 1$, then the expression becomes

$$r^{k_0}(f r^{k_1}) f^{l_1} \cdots = r^{k_0}(r^{n-k_1} f) f^{l_1} \ldots$$

which we can simplify by combining ff^{l_1}. This process can be repeated until we reach the form $r^k f^\ell$ for $0 \leq k < n$ and $\ell \in \{0, 1\}$. □

(c) Prove that the order (number of elements) of the group is $D_n = 2n$.

Proof. Now that we have established that all elements of D_n can be written as the form $r^k f^\ell$ for $0 \leq k < n$ and $\ell \in \{0, 1\}$, this queestion can be reframed as asking "How many unique ways can we write $r^k f^\ell$?" By combinatorics, we see that the number of unique ways is $2n$. □

2. We say that a group $(G, *)$ is an abelian group if and only if $a * b = b * a$ for all $a, b \in G$.

(a) Prove that if G is generated by g_1 and g_2 then G is abelian if and only if $g_1 * g_2 = g_2 * g_1$.

Proof. First, we prove that if $a * b = b * a$ for all $a, b \in G$, then $g_1 * g_2 = g_2 * g_1$. By setting $a = g_1$ and $b = g_2$, then $g_1 * g_2 = g_2 * g_1$.
The converse "if $g_1 * g_2 = g_2 * g_1$, then $a * b = b * a$ for all $a, b \in G$" is much more difficult to prove. Notice that

$$a = a_1 * a_2 * \cdots * a_p = \prod_{i=1}^{p} a_i \tag{3}$$

$$b = b_1 * b_2 * \cdots * b_q = \prod_{i=1}^{q} b_i \tag{4}$$

$$c = a * b = a_1 * a_2 * \cdots * a_p * b_1 * b_2 * \cdots * b_q \tag{5}$$

where $a_i, b_i \in \{g_1, g_2\}$.
Notice that because $g_1 * g_2 = g_2 * g_1$, we can swap any two adjacent terms of c.
Therefore, we can rewrite c as

$$c = a * b = b_1 * b_2 * \cdots * b_q * a_1 * a_2 * \cdots * a_p. \tag{6}$$

Therefore, G is generated by g_1 and g_2 then G is abelian if and only if $g_1 * g_2 = g_2 * g_1$. □

(b) Suppose G is generated by $g_1, g_2, \ldots, g_n$. Do you think that G is abelian if and only if $g_i * g_j = g_j * g_i$ for all $i, j \in \{1, 2, \ldots, n\}$? Explain your reasoning, but you do not have to provide a proof.
I do think so. We can likely apply some inductive proof to show that adding a new g_k that is mutually commutative with all other g_i does not impact the 'abelian-ness' of the overall group G.

(c) Prove that cyclic groups C_n are abelian.

Proof. By definition,

$$C_n = \langle r \mid r^n = e \rangle.$$

Because the cyclic groups C_n are defined by a generator with one non-identity element, then C_n must be abelian by associativity.

$$r^k * r^l = (\underbrace{r * r * \cdots * r}_{k}) * (\underbrace{r * r * \cdots * r}_{l}) \tag{7}$$

$$= (\underbrace{r * r * \cdots * r}_{l}) * (\underbrace{r * r * \cdots * r}_{k}) \tag{8}$$

$$= r^l * r^k. \tag{9}$$

□

(d) Prove that the general linear group of 2×2 real-valued matrices

$$GL_2(\mathbb{R}) = \{A \in \mathbb{R}^{n \times n} \mid \det(A) \neq 0\}$$

is not abelian.

Proof. Let

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, B = \begin{bmatrix} p & q \\ r & s \end{bmatrix}.$$

Then, $A \times B = \begin{bmatrix} ap + br & aq + bs \\ cp + dr & cq + ds \end{bmatrix}$ and $B \times A = \begin{bmatrix} ap + cq & bp + dq \\ ar + cs & br + ds \end{bmatrix}$.
In general, these are not the same matrix. □

(e) Prove that the dihedral group D_n is abelian if and only if $n \leq 2$.

Proof. First, by definition, the smallest dihedral group is D_1, which is just the identity. This trivial group must be abelian.
Then, for $n = 2$,

$$D_2 = \langle r, f \mid r^2 = f^2 = e \rangle.$$

Notice that in this case, $r = f$, therefore, D_2 is abelian.
Then, to prove the "other direction", we must prove that a dihedral group D_n being abelian implies that $n \leq 2$. We can prove this via contrapositive. Thus, we for a dihedral group D_n where $n > 2$, D_n is not abelian.
Suppose we have D_n where $n > 2$. We choose the element $rf \in D_n$. Now, $rf = fr^{n-1} \neq fr$, so D_n is not abelian.
Therefore, D_n is abelian if and only if $n \leq 2$. □

3. The direct product of groups is a way of combining two groups into a new group. The notation can look a bit intimidating, but the basic idea is simple: just treat the groups independently. Here's the formal definition, but keep in mind that $*$, $\star$, and $\odot$ are just names for binary operations like $+$, $\times$, and $\circ$.

Given two groups $(G_1, *)$ and $(G_2, \star)$, we define the direct product of groups as

$$G_1 \times G_2 = \{(g_1, g_2) \mid g_1 \in G_1 \text{ and } g_2 \in G_2\}$$

with the binary operation, $\odot$, where

$$(g_1, g_2) \odot (g_1', g_2') = (g_1 * g_1', g_2 \star g_2').$$

(a) Let $(G_1, *) = (D_4, *)$ be the dihedral group of the square, and let $(G_2, \star) = (S_3, \circ)$ be the symmetric group under function composition. Show that in $G_1 \times G_2$,

$$(f, (132)) \odot (r, (23)) = \left(r^3 f, (13)\right).$$

$$(f, (132)) \odot (r, (23)) = (fr, (132) \circ (23)) \tag{10}$$
$$= (r^3 f, (2)(31)) = (r^3 f, (13)). \tag{11}$$

(b) Prove that for any two groups G_1 and G_2, the direct product of groups is a group.

Proof. Let $G = G_1 \times G_2$. We verify that G follows the group axioms:

Closure: Let $g_1, g_1' \in G_1$ and $g_2, g_2' \in G_2$. since $g_1 g_1' \in G_1$, and $g_2 g_2' \in G_2$, $(g_1 g_1', g_2, g_2') \in G$.

Identity: Let e_1, e_2 be the identity elements of G_1 and G_2 respectively. (e_1, e_2) is the identity of G.

Associativity: Since the operations on the first and second elements of G are both associative, associativity is inherited to the direct product.

Inverse: Similarly, because the left and right elements each independently have an inverse, the tuple containing both will be the equivalent inverse in G.

□

(c) Prove that if $G_1 \times G_2$ is abelian if and only if G_1 and G_2 are both abelian.
First, if $G = G_1 \times G_2$ is abelian, then for $(a, b), (a', b') \in G$,

$$(a, b) \odot (a', b') = (a', b') \odot (a, b) \tag{12}$$
$$(a * a', b \star b') = (a' * a, b' \star b) \tag{13}$$

Since $a * a' = a' * a$ and $b \star b' = b' \star b$, G_1 and G_2 are abelian.
Now, given G_1 and G_2 are abelian, reverse the above algebra to see that G must also be abelian.
Thus, $G_1 \times G_2$ is abelian if and only if G_1 and G_2 are both abelian.

(d) Let $\mathbb{Z}/k\mathbb{Z}$ denote the group of the integers mod k under addition. Let $G = \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/3\mathbb{Z}$. Compute the order of G. Prove that G has an element of order $|G|$.

The order of a direct product of two groups is the product of the orders. Thus,

$$|G| = |\mathbb{Z}/2\mathbb{Z}| \times |\mathbb{Z}/3\mathbb{Z}| = 2 \times 3 = 6.$$

The element with order 6 is $(1, 1)$.

(e) Let $G = \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$. Compute the order of G. Prove the G has no element of order $|G|$.

Similarly, $|G| = 2 \times 2 = 4$. Now, the elements of G are $\{(0, 0), (0, 1), (1, 0), (1, 1)\}$. The orders of all of these elements is 2.

Exercise 3 [1]

My favorite resource from this class is OpenSCAD. Although I will likely never use it again, it was very fun to play around with. The backstory for the development of it is also really cool.

Choice Exercise 6 [10]

1. [3] Write a function that takes a word in S_n and outputs it in cycle notation. For instance, if the word is $\pi = 156243 \in S_n$ then the input to your program would be [1,5,6,2,4,3], and since $\pi = (254)(36)$ in cycle notation, the output would be [[254],[36]] or something equivalent.

```
word = [0,1,5,6,2,4,3] # prepend with a zero for convenience.

def word_to_cycle(word):
    explored = set()
    def explore(i):
        out = []
        while i not in explored:
            explored.add(i)
            out.append(i)
            i = word[i]
        return out

    out = [explore(i) for i in word]
    return list(filter(lambda x: len(x) > 1, out))


print(word_to_cycle(word)) # [[5, 4, 2], [6, 3]]
```

   I was unfortunately not able to write this in Scheme LISP. I was defeated, but I managed to solve the other way

2. [2] Write a function that goes the other way: it takes in input of a permutation in cycle notation (e.g. [[254],[36]]) and outputs the corresponding permutation as a word (e.g. [1,5,6,2,4,3].)

```
(define (max-vec v)
  (define (max-iter v champ i)
    (if (= i (vector-length v))
        champ
        (if (> (vector-ref v i) champ)
            (max-iter v (vector-ref v i) (+ i 1))
            (max-iter v champ (+ i 1)))))

  (max-iter v (vector-ref v 0) 0))

(define (max-vec-vec vv)
  (define (max-iter vv champ i)
    (if (= i (vector-length vv))
        champ
        (let ((m (max-vec (vector-ref vv i))))
          (if (> m champ)
              (max-iter vv m (+ i 1))
              (max-iter vv champ (+ i 1))))))
  (if (= (vector-length vv) 0)
      (error "Empty vector")
      (max-iter vv (max-vec (vector-ref vv 0)) 1)))

(define (cycle-to-word cycles)
    (define m (+ 1 (max-vec-vec cycles)))
    (define word (make-vector m 0))
    (define (cycle-iter cycle i)
```

```
        (define n (vector-length cycle))
        (if (< i n)
            (begin
                (vector-set!
                    word
                    (vector-ref cycle i)
                    (vector-ref cycle (modulo (+ 1 i) n)))
                (cycle-iter cycle (+ 1 i))
            )))
    (define l (vector-length cycles))
    (define (cycles-iter i)
        (if (< i l)
            (begin
                (cycle-iter (vector-ref cycles i) 0)
                (cycles-iter (+ 1 i))
            )
        )
    )
    (cycles-iter 0)
    word
)
```

3. [2] Write a function that takes two permutations as words and composes them, working right-to-left as usual.

```
word1 = [0,6,2,1,4,3,5]
print(word, word1)

def compose_word(a, b):
    out = [0] * max(len(a), len(b))

    for i in range(1, len(out)):
        out[i] = b[a[i]] if a[i] < len(b) else a[i]

    return out

print(compose_word(word, word1)) # [0, 6, 3, 5, 2, 4, 1]
```

4. [2] Write a function that takes two permutations in cycle notation and composes them, working right-to-left as usual.

```
def cycle_to_word(cycles):
    n = 1 + max(max(cycle) for cycle in cycles)
    word = list(range(n))

    for cycle in cycles:
        for x, y in zip(cycle, cycle[1:] + [cycle[0]]):
            word[x] = y

    return word

print(cycle_to_word(word_to_cycle(word))) #[0,1,5,6,2,4,3]

def compose_cycle(a, b):
    x = cycle_to_word(a)
```

```
    y = cycle_to_word(b)
    return word_to_cycle(compose_word(x, y))

p1 = [[2,5,4],[3, 6]]
p2 = [[4,5], [1,2]]
print(compose_cycle(p1, p2)) #[0, 2, 4, 6, 1, 5, 3]
```

5. [1] Write two functions that compute the order of a permutation, one that works on words, and one that works on cycle notation.

```
def order_cycle(cycles):
    curr = cycles
    i = 1

    while len(curr) > 0:
        i += 1
        curr = compose_cycle(cycles, curr)

    return i

def order_word(word):
    curr = word
    i = 1
    while not all(index == x for (index, x) in enumerate(curr)):
        i += 1
        curr = compose_word(word, curr)

    return i

print(order_cycle(p1))
print(order_word(cycle_to_word(p1)))
```